BÁO CÁO THỰC HÀNH

LOS201 - EMBEDDED LINUX

LAB 02

Chủ đề: Character Device Driver, PROCFS, SYSFS và JFFS2

Thông tin	Nội dung
Họ và tên	Doan Truong Son
Mã sinh viên	SE200908
Kernel	Linux 5.15.0
Kiến trúc	ARMv7 - ARM EABI5
Compiler	arm-linux-gnueabihf-gcc 15.2.0
Driver	lab2_driver.ko

Năm học: .....

1. MỤC TIÊU BÀI THỰC HÀNH

Bài thực hành LAB02 giúp sinh viên tìm hiểu quy trình xây dựng Character Device Driver trong Linux Kernel.

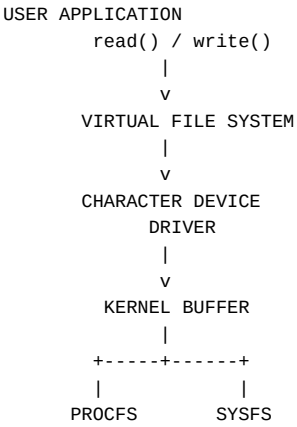
Các mục tiêu chính:

- Xây dựng và biên dịch Linux Kernel Module.
- Đăng ký Character Device Driver.
- Trao đổi dữ liệu giữa Kernel Space và User Space.
- Sử dụng PROCFS và SYSFS.
- Tìm hiểu MTD và hệ thống tập tin JFFS2.
- Tích hợp driver vào Root Filesystem.

2. MÔI TRƯỜNG THỰC HIỆN

Thành phần	Thông tin
Kernel	Linux 5.15.0
Architecture	ARMv7
ABI	EABI5
Compiler	arm-linux-gnueabihf-gcc 15.2.0
Module	lab2_driver.ko
Root filesystem	initramfs
Filesystem	JFFS2
vermagic	5.15.0 SMP mod_unload ARMv7 p2v8

3. KIẾN TRÚC CHARACTER DEVICE DRIVER



Ứng dụng User Space giao tiếp với driver thông qua device node. VFS chuyển yêu cầu đến các hàm callback được đăng ký trong cấu trúc file_operations.

Các hàm thường gặp gồm open(), read(), write() và release(). Driver xử lý yêu cầu, kiểm tra dữ liệu và thực hiện trao đổi với vùng nhớ trong Kernel Space.

3.1. Trao đổi dữ liệu

copy_from_user() sao chép dữ liệu từ User Space vào Kernel Space. copy_to_user() sao chép dữ liệu từ Kernel Space về User Space.

Không sử dụng memcpy() trực tiếp với con trỏ User Space vì kernel không thể mặc định truy cập an toàn vùng nhớ của tiến trình. Các hàm copy_from_user() và copy_to_user() hỗ trợ kiểm tra lỗi trong quá trình sao chép.

3.2. Đồng bộ dữ liệu

DEFINE_MUTEX(lab2_mutex) khai báo mutex để bảo vệ vùng dữ liệu dùng chung. Mutex giúp tránh Race Condition khi nhiều tiến trình truy cập driver đồng thời.

4. PROCFS VÀ SYSFS

PROCFS: Hệ thống tập tin ảo dùng để cung cấp thông tin trạng thái hệ thống và kernel.

SYSFS: Giao diện thể hiện các đối tượng thiết bị, driver và thuộc tính của chúng.

4.1. Kiểm tra PROCFS

```
cat /proc/lab2_info
```

[BỔ SUNG LOG PROCFS THỰC TẾ VÀO ĐÂY]

4.2. Kiểm tra SYSFS

```
find /sys -iname '*lab2*' 2>/dev/null
```

[BỔ SUNG ĐƯỜNG DẪN VÀ LOG SYSFS THỰC TẾ VÀO ĐÂY]

5. KIỂM TRA CHARACTER DEVICE DRIVER

```
modinfo lab2_driver.ko
insmod lab2_driver.ko
lsmod | grep lab2
ls -l /dev/lab2
dmesg | grep -i lab2
/usr/bin/test_driver.sh
```

Các lệnh trên dùng để kiểm tra thông tin module, nạp driver, xác nhận module đã được nạp và kiểm tra thông báo kernel.

[BỔ SUNG LOG CHẠY DRIVER THỰC TẾ VÀO ĐÂY]

6. HỆ THỐNG TẬP TIN MTD VÀ JFFS2

MTD là lớp con của Linux Kernel cung cấp giao diện quản lý các thiết bị nhớ flash.

JFFS2 là hệ thống tập tin được thiết kế cho thiết bị flash dạng raw NAND/NOR.

6.1. Kiểm tra MTD

```
cat /proc/mtd
mtdinfo <THIET_BI_MTD>
```

[BỔ SUNG LOG MTD THỰC TẾ VÀO ĐÂY]

6.2. Thao tác JFFS2

Các thao tác dưới đây được thực hiện trên HOST Ubuntu. Phải xác định đúng thiết bị MTD trước khi xóa hoặc ghi dữ liệu.

```
# Xac dinh thiet bi MTD
cat /proc/mtd

# Thay <THIET_BI_MTD> bang thiet bi dung
flash_erase <THIET_BI_MTD> 0 0
nandwrite <THIET_BI_MTD> jffs2.img

mkdir -p /mnt/jffs2
mount -t jffs2 <THIET_BI_MTD> /mnt/jffs2

ls -lah /mnt/jffs2
cat /mnt/jffs2/hello.txt

umount /mnt/jffs2
```

[BỔ SUNG LOG FLASH_ERASE, NANDWRITE, MOUNT, ĐỌC FILE, REMOUNT VÀ NANDDUMP THỰC TẾ VÀO ĐÂY]

7. SO SÁNH CÁC HỆ THỐNG TẬP TIN

Filesystem	Đặc điểm	Ưu điểm	Hạn chế
ext4	Block device	Ổn định, phổ biến	Không chuyên cho raw NAND
JFFS2	Raw flash	Phù hợp flash dung lượng nhỏ	Mount có thể chậm khi nhiều node
UBIFS	Raw NAND qua UBI	Quản lý NAND tốt	Cấu hình phức tạp hơn
SquashFS	Read-only	Nén tốt, tiết kiệm dung lượng	Không ghi trực tiếp
cramfs	Read-only	Đơn giản, nhỏ gọn	Ít linh hoạt

7.1. Đề xuất cho thiết bị IoT có NAND 32 MB

Đối với thiết bị IoT có NAND 32 MB, có thể sử dụng SquashFS cho Root Filesystem chỉ đọc nhằm tiết kiệm dung lượng.

Một phân vùng JFFS2 hoặc UBI/UBIFS có thể được sử dụng cho dữ liệu cần ghi thay đổi thường xuyên. Lựa chọn cuối cùng phụ thuộc vào dung lượng thực tế, erase block, RAM, kernel và bootloader.

8. CÂU HỎI PHÂN TÍCH

Câu 1. Vì sao không dùng memcpy() giữa kernel và user buffer?

Vùng nhớ User Space không được kernel truy cập trực tiếp tùy ý. copy_from_user() và copy_to_user() cung cấp cơ chế truy cập an toàn hơn và xử lý lỗi khi sao chép dữ liệu.

Câu 2. Vì sao sử dụng DEFINE_MUTEX(lab2_mutex)?

Mutex bảo vệ tài nguyên dùng chung khi nhiều tiến trình truy cập driver đồng thời. Điều này giúp tránh Race Condition và đảm bảo tính nhất quán của dữ liệu.

Câu 3. So sánh PROCFS và SYSFS.

PROCFS thường cung cấp thông tin trạng thái hệ thống và kernel. SYSFS thể hiện các đối tượng thiết bị, driver và thuộc tính của chúng. SYSFS có cấu trúc rõ ràng cho mô hình thiết bị.

Câu 4. Vì sao JFFS2 cần erase trước khi ghi?

NAND Flash không cho phép ghi đè tùy ý lên dữ liệu cũ. Muốn tái sử dụng erase block cần xóa block theo quy tắc phần cứng. Trong khi đó ext4 hoạt động trên block device và ghi theo giao diện block/sector.

Câu 5. Nếu insmod trong rcS thất bại, boot có tiếp tục không?

Điều này phụ thuộc cách viết rcS và shell. Nếu script không dừng ngay khi lệnh lỗi và không gọi exit, các lệnh tiếp theo có thể vẫn chạy. Tuy nhiên driver sẽ không hoạt động. Cần kiểm tra mã trả về và xử lý lỗi phù hợp.

9. KHÓ KHĂN VÀ CÁCH KHẮC PHỤC

Điền ít nhất hai khó khăn thực tế đã gặp trong quá trình thực hiện bài lab. Không sử dụng nội dung minh họa thay cho trải nghiệm thực tế.

Khó khăn	Nguyên nhân	Cách khắc phục
[Khó khăn thực tế 1]	[Mô tả lỗi]	[Cách xử lý và kết quả]
[Khó khăn thực tế 2]	[Mô tả lỗi]	[Cách xử lý và kết quả]

10. CẤU TRÚC THƯ MỤC

```
LAB02/
|-- README.md
|-- driver/
|   |-- lab2_driver.c
|   |-- Makefile
|   |-- lab2_driver.ko
|   |-- sms_assignment/
|   `-- task2_misc/
|-- configs/
|-- logs/
|-- mtd/
|   |-- jffs2.img
|   |-- nand_dump.bin
|   `-- jffs2_data/
|-- output/
|   `-- initramfs_lab2.cpio.gz
|-- rootfs/
|   `-- initramfs/
|       |-- etc/init.d/rcS
|       |-- lib/modules/5.15.0/
|       `-- usr/bin/
`-- bao_cao/
    `-- SE200908_Lab02_BaoCao.pdf
```

11. KẾT LUẬN

Thông qua bài thực hành LAB02, sinh viên tìm hiểu được quy trình xây dựng Linux Character Device Driver, trao đổi dữ liệu giữa Kernel Space và User Space, sử dụng PROCFS, SYSFS và hệ thống tập tin JFFS2.

Kết quả thực nghiệm và log chạy thực tế cần được bổ sung đầy đủ để chứng minh các chức năng đã được kiểm tra.